

Project 1: Counter App

0



After 2 clicks in **increase**:

2



Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Counter Apps</title>
  <link rel="stylesheet" href="style.css">
  <script src="fun.js"></script>
</head>

<body>
  <h1 id="counter">0</h1>
```

```
    <button onclick="increase()">Increase</button>
<button onclick="reset()">Reset</button>
    <button onclick="decrease()">Decrease</button>
</body>
</html>
```

Style.css

```
body {
    text-align: center;
    margin-top: 50px;
}

button {
    margin: 5px;
    padding: 10px 25px;
    font-size: 16px;
}

h1 {
    font-size: 48px;
}
```

fun.js

```
let count = 0;
function increase(){
    count++;
    updateCounter();
}
```

```
function decrease() {  
  count--;  
  updateCounter();  
}  
  
function reset() {  
  count = 0;  
  updateCounter();  
}  
  
function updateCounter() {  
  document.getElementById("counter").innerText = count;  
}
```

What is Synchronous JavaScript?

In synchronous programming, operations are performed sequentially, one after the other. So, each line of code waits for the previous one to finish before proceeding to the next. This means that the program executes in a predictable, linear order, with each task being completed before the next one starts.

Example: In this example, we have shown the synchronous nature of JavaScript.

```
console.log("First Line");  
console.log("Second Line");  
console.log("Third Line");  
console.log("Fourth Line");  
console.log("Fifth Line");
```

What is Asynchronous JavaScript?

Asynchronous programming, on the other hand, enables multiple tasks to run independently of one another. In asynchronous code, a task can be initiated, and while waiting for it to complete, other tasks can proceed. This non-blocking nature helps improve performance and responsiveness, especially in web applications.

Example: In this example, we have shown the Asynchronous nature of JavaScript.

```
console.log("First Line");  
console.log("Second Line");  
setTimeout(() => {  
  console.log("Third Line");  
}, 3000);
```

```
console.log("Fourth Line");  
setTimeout(() => {  
  console.log("Fifth Line");  
}, 2000);
```

Output:

First Line

Second Line

Fourth Line

Fifth Line

Third Line

Project using Asynchronous: Simulate fetching student results from a server using asynchronous JavaScript.

Concept:

- Display basic info instantly
- Fetch results with delay (like a real server)
- Show results when loaded

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <title>Document</title>
  <script src="script.js"></script>
</head>
<body>

<h2>Student Result Portal</h2>
<button onclick="getResult()">Get Result</button>

<p id="status"></p>
<p id="result"></p>

</body>
</html>
```

script.js

```
function getResult() {  
    document.getElementById("status").innerText =  
    "Fetching result...";  
  
    console.log("Request sent to server");  
  
    setTimeout(() => {  
        document.getElementById("status").innerText =  
        "Result Loaded!";  
        document.getElementById("result").innerText =  
        "Name: Rahim | CGPA: 3.75";  
  
        console.log("Result received");  
    }, 3000);  
  
    console.log("User can continue browsing");  
}
```